UniMind: Calibration-Aware Scheduling and Placement for Quantum-Accelerated AI Middleware

Y. X. Tan

***Abstract*—Quantum-accelerated AI workloads must be scheduled onto physical devices whose calibration changes between runs. A classical scheduler assumes resources are identical and stable; a quantum scheduler faces an array of qubits whose individual noise parameters drift daily. This paper is a *measurement study* of how a quantum runtime should maintain scheduling decisions when hardware calibration changes over time. We instrument a 156-qubit superconducting device (`ibm_marrakesh`) across four anchored daily snapshots (D0–D3; 2026-08-29 to 09-01) and, to bound the validity window, 337 hourly calibration snapshots across three backends (`marrakesh` 111, `kingston` 109, `fez` 117) from 71 calibration epochs (2026-08-31 to 09-02, calibration-only). We build UniMind, a middleware that scores, places, executes, and refreshes calibration data in a closed loop. Four findings are established: (1) Calibration validity is temporally bounded and small. Day-over-day rank correlation is 0.567–0.579 (Kendall τ=0.42–0.43), top-10 overlap is 0.11–0.25 across all six D0–D3 pairs, and a stale top-3 pin is 1.45–$2.55\times$ worse than a fresh selection (1.94–$2.1\times$ conservative QPU-anchored D0→D1/D2, 24–34 h). Hourly telemetry shows the same drift mechanism—tiny top-3 margins ($\sim 10^{-3}$) versus median $|\Delta C| = 0.0127$—explaining daily turnover while intra-epoch hourly stability is perfect (0/71 epochs fire the trigger, $S(t)$=1.0 to 15 h, 337 total/328 in KM, 3 backends, calibration-only), bounding $T_{\rm valid}$ from both sides. We formalize the *Calibration Validity Horizon* $T_{\rm valid}$ as a lower bound design primitive for quantum runtimes. (2) Adaptive refresh dominates static and periodic policies at near-zero cost. A two-line trigger (top-10 Jaccard < 0.5 or ΔC>0.005) detects staleness in ~ 0.04 ms per scoring pass. Offline sweeps over 144 (τ_J, τ_C) thresholds and explicit static/periodic/adaptive simulation (periodic 6h fires $5\times$, 48h misses the drift) place the default trigger at the Pareto knee. (3) Refresh, not placement selection, dominates end-to-end reliability at the measured scale. Fresh pins lift *both* the pinned and free arms by $+33/37$ pp on a 6-qubit angle suite ($n = 30$/arm, power analysis: MDE ≈ 32 pp at 80% power); the pinned vs. free difference is within noise (70.0% vs. 76.7%, p=0.56) and under-powered by design, reported as a bounded null. (4) Proxy-trained layout models do not transfer. Re-fitting the 7-dim layout utility $J(G)$ on real device labels collapses its size out-of-sample Spearman from ρ=0.818 to 0.13 (p=0.36); feature ablation and connectivity-vs-quality variance analysis show the simulator proxy over-penalizes large layouts that a refresh-pinned device does not, and that beyond $k \approx 100$ connectivity dominates quality variance—exactly where the proxy errs. We give the validity model, the measured numbers, the adaptive mechanism, and the explicit negative results.**

***Index Terms*—quantum computing middleware, calibration-aware scheduling, calibration validity horizon, adaptive refresh, qubit placement, calibration drift, resource management, quantum-cloud runtime, reliability composition.**

I. Introduction

Y. X. Tan is an independent researcher (e-mail: yxtan5022@gmail.com). All experiments are open-source and reproducible from `yxtan5022-maker/unimind-v2`; QPU job IDs and snapshots are recorded in the data directory.

EMBEDDED AI is increasingly offloaded to accelerator hardware, and quantum processors are the newest such accelerator. Their software stack, however, changes the *scheduler's* job in a fundamental way: a quantum circuit's fidelity depends on *which* physical qubit executes it and *when* that qubit was last calibrated [1], [2], [5], [6]. A classical scheduler assumes resources are identical and stable; a quantum scheduler faces an array of hundreds of qubits whose individual noise parameters move between runs [10], [13], [17].

Quantum-machine-learning (QML) frameworks—PennyLane [4], Qiskit [18], TensorFlow Quantum—provide algorithm abstractions but not the *system* layer: dynamic backend discovery, layout selection, and calibration refresh. Applications are assumed, usually silently, to have been placed on "good enough" qubits. On real devices this assumption fails measurably: with *free* transpiler placement an angle-encoding experiment on `ibm_marrakesh` missed the 0.05 fidelity tolerance by up to 0.392, while the *same* circuits on a pinned calibrated layout passed everywhere (max. deviation 0.028) [25]. Placement is not cosmetic; it is the dominant accuracy term at our test scale.

This paper addresses the temporal dimension of that problem. Placement decisions are made from a calibration snapshot, and snapshots change daily; the scheduler's real questions are "*how old is my snapshot*" and "*should I refresh*" as much as "*where to place*". We define the **Calibration Validity Horizon** $T_{\rm valid}$: the maximum age of a calibration snapshot such that the resulting placement remains within fidelity tolerance. With $T_{\rm valid}$ small (hours, not days), the runtime must refresh frequently; with $T_{\rm valid}$ large, refresh is wasteful. We measure a *lower bound* on $T_{\rm valid}$ on a 156-qubit device across four anchored daily snapshots (D0–D3, plus 337 hourly snapshots across three backends for the drift mechanism), design an adaptive refresh trigger that detects staleness at near-zero cost, and show that refresh—not placement selection—is where reliability engineering should invest at the measured scale. The middle section of the paper is a deliberately adversarial audit of our own utility function, because the positive result (§VII, §VIII) is precisely the one the optimistic number (proxy-trained ρ=0.82) would have hidden.

A. Contributions

All QPU claims are measured on `ibm_marrakesh` (156 qubits, heavy-hex); four daily snapshots (D0–D3, 2026-08-29 to 09-01) anchor calibration, three (D0–D2) anchor every QPU job, and 337 hourly snapshots from 71 calibration epochs (2026-08-31 to 09-02, `marrakesh` 111 + `kingston` 109 + `fez` 117) bound the drift mechanism without additional QPU cost. We make four contributions:

1) **Temporal validity analysis as a lower bound.** We quantify day-over-day drift (ρ=0.567–0.579, τ=0.42–0.43, top-10 Jaccard 0.11–0.25 across all D0–D3 pairs; D0→D1 ρ=0.579, J_{10}=0.25), measure the staleness cost of aged pins (stale top-3 is 1.45–2.55× worse fresh; 1.94–2.1× conservative across D0→D1/D2, 24–34 h, Table XI), formalize the Calibration Validity Horizon T_{valid} (Def. 2) as a *lower-bound* design primitive, explain turnover via ranking margins ($M_3 \sim 10^{-3}$ vs. median $|\Delta C| = 0.0127$, §VII), and bound it hourly: 0/71 epochs fire intra-epoch up to 15 h (337 snapshots, 3 backends), while every inter-day transition fires (J_{10}=0.11–0.25).
2) **Adaptive refresh policy with explicit baselines.** We design a trigger (Jaccard $<$ 0.5 or ΔC>0.005) that detects staleness in $\sim$ 0.04 ms, run an offline sweep over 144 (τ_J, τ_C) thresholds, and simulate static, periodic (6/12/24/48h), and adaptive policies on the same D0→D1 transition (§VII-D–§VII-E); the default trigger fires exactly once and restores +33 pp, while periodic-6h fires 5× for the same drift.
3) **Bounded placement results.** The 7-dim layout utility $J(G)$, trained on a simulator proxy and re-fitted on real device labels, collapses from size-OOS ρ=0.818 to 0.13 (p=0.36); at k=6 with fresh pins, pinned placement shows no measured advantage over the free transpiler (70.0% vs. 76.7%, p=0.56, MDE $\approx$ 32 pp at 80% power). Feature ablation and the connectivity-vs-quality variance crossover at $k \approx$ 100–128 explain the mechanism (§VI, §VIII).
4) **Measurement methodology.** Every number traces to a recorded job or snapshot (§X); where an exploratory result did not replicate (ρ=1.000 → 0.771, p=0.103) we report the failure; all proportions use Wilson intervals and two-sided Fisher exact tests, and we report minimum detectable effects where we claim a bounded null.

Scope and framing. This is a measurement study, not a claim of quantum advantage, novel quantum algorithms, or a production runtime. We propose T_{valid} as a design primitive; with four daily snapshots (three QPU-anchored) plus 337 hourly calibration snapshots from 71 epochs across three backends we establish its lower bound and mechanism on one device family (heavy-hex), but leave the full multi-device QPU validation and workload-general k-dependent decay curve to future work. The LLM-orchestration layer is a controlled-parameter reliability model (§VIII-A, App. D), not a claim about real LLM quality, and is separated from the calibration-aware scheduling core so the middleware contribution can be evaluated independently.
Organization. Section III formalizes the scheduling problem and surveys related work; §IV describes the middleware; §V–VI establish the placement results and their limits; §VII measures drift and validates the refresh policy; §VIII composes the software and hardware paths; §IX states what the measurements do and do not say. Table I fixes notation.

TABLE I
NOTATION

Symbol	Meaning
S_t	calibration snapshot at time t (readout errors, gate errors, T_1, T_2)
$C(q)$	total readout assignment error $p_{0\|1}(q) + p_{1\|0}(q)$
ρ, τ	Spearman, Kendall rank correlation across $S_t \to S_{t'}$
k	number of logical qubits of a circuit
$\pi : \hat{G} \to Q$	placement of the logical circuit on physical qubits
w	amplitude weight of one encoded value; $\theta = 2 \arcsin \sqrt{w}$
$Z_{\text{emp}}, Z_{\text{th}}$	empirical, theoretical Z-measurement
ϵ	fidelity tolerance (0.05); pass iff $\max_i \|Z_i^{\text{emp}} - Z_i^{\text{th}}\| \le \epsilon$
$P_{\text{obs}}(1) = b + a\,w$	two-parameter affine readout channel model
$J(G)$	7-dim layout utility with weights $(\alpha, \beta, \gamma, \gamma_{\log}, \eta_{\text{idle}}, \delta, \lambda)$
N_{SWAP}, D	inserted SWAPs and transpiled depth
g, r, f	per-call orchestrator outcome probs. (correct, broken, unavailable)
H, K	healing budget, generation-call budget
$R_{\text{end}}, \rho_{\text{heal}}, c$	end-to-end reliability, healed-path reliability, fallback coverage

II. Experimental Setup

A. Device and snapshots

All QPU results use the IBM Quantum 156-qubit superconducting device `ibm_marrakesh` (heavy-hex topology, 176 couplers). We anchor every device experiment to three full calibration snapshots—D0, D1, D2—at roughly 24-hour and 10-hour spacing and run every device experiment against the matching snapshot (Table II). A fourth daily snapshot D3 (2026-09-01, `calib_2026-09-01.json`) is calibration-only and extends the daily drift matrix to 4×4 pairs. Two snapshot families recur: the *pinned* family `calib_2026-08-{29,30,31}.json` + D3, and a legacy 08-23 capture used only for the placement anchor of §V. Where "stale" pins are evaluated, the pins are always taken from D0 while execution happens on D1/D2/D3-era device state.

To bound the drift mechanism without additional QPU cost we also analyze 337 hourly calibration snapshots from 71 calibration epochs across `ibm_marrakesh` (111), `ibm_kingston` (109) and `ibm_fez` (117), 2026-08-31 to 09-02 (`data/calib_snapshots/`, `telemetry_log.jsonl`). These are calibration-only (no jobs) and are used in §VII to quantify ranking-margin turnover and hourly stability (survival analysis, 0/71 false positives); they do not change any QPU job label.

B. Common protocol

The instrument circuit is angle (RY) encoding on Z-measurement, 19-weight grid w=0.05, …, 0.95 for the single-qubit sweeps and 30 parameterizations for the 6-qubit batches; transpiler seed 42 throughout; layouts are pinned via `initial_layout` where placement is under test and left free where it is the baseline. Shots are 8192 (single-qubit sweeps) or 4096 (multi-qubit suites). The pass criterion is per-bit $\max_i |Z_i^{\text{emp}} - Z_i^{\text{th}}| \le \epsilon$ with ϵ=0.05, fixed before

TABLE II
SNAPSHOT TIMELINE. PINS FOR EACH EXPERIMENT FAMILY REFERENCE THE STATED SNAPSHOT; EXECUTION TIMES ARE THE JOB QUEUE WINDOWS RECORDED IN THE DATA FILES. HOURLY TELEMETRY (71 EPOCHS, 337 SNAPSHOTS, 3 BACKENDS) IS CALIBRATION-ONLY.

Label	device update	capture	used as pins for
legacy 08-23	2026-08-23	2026-08-23	§V anchor
D0	2026-08-29 21:26	22:07	§V-B, stale arms
D1	2026-08-30 21:29	23:03	drift/staleness eval
D2	2026-08-31 07:46	07:46*	refresh arms (§VI,§VIII)
D3	2026-09-01 23:19	00:05+1	drift matrix ext. (calib-only)
hourly	2026-08-31–09-02	337× /71 epochs	drift mechanism + survival

data release. All statistical tests are two-sided unless stated; proportions use Wilson 95% intervals and two-sided Fisher exact tests computed from the raw pass counts (two-proportion z where noted). For claimed bounded nulls we report minimum detectable effect (MDE) at 80% power. Details that would be tedious in the main text (per-row values) are in the appendix and in `analysis/results/*.json`.

III. PROBLEM AND RELATED WORK

A. Formal model

Definition 1 (Calibration-aware scheduling problem): A device is a set of qubits Q and couplers E with a calibration snapshot S_t reporting, per qubit, readout assignment errors $(p_{0|1}, p_{1|0})$, gate errors, and coherence times (T_1, T_2). A job is a logical circuit $\hat{G}$ over k logical qubits. The scheduler chooses a *placement* $\pi : \hat{G} \to Q$ (with $|\pi| = k$, connected in (Q, E) for our chains), a transpiler configuration, and a *snapshot age policy* that decides when S_t is re-fetched. Its objective, subject to placement, snapshot age, and overhead budgets, is per-qubit measurement fidelity $\max_i |Z_i^{\text{emp}} - Z_i^{\text{th}}| \leq \epsilon$ at a reliability target.

The distinctness from classical placement lies in (Q, E, S_t) being jointly non-stationary: π chosen at time t is evaluated at execution time $t' > t$ against a changed $S_{t'}$. The paper's arithmetic is about the size of that violation. Three assumptions bound the model.

- **(A1) Readout dominates measurement.** For the Z-measurement instrument used throughout, per-qubit readout assignment error is the leading distortion term; gate noise is handled by the placement/transpiler and by coupler terms in $J(G)$, but readout is the mechanism we isolate first.
- **(A2) Snapshots are the operative truth.** The scheduler sees the device only through calibration snapshots; it cannot poll the device state at shot granularity.
- **(A3) Refresh is cheap.** A snapshot fetch and a full-156 scoring pass cost ≈ 0.04–0.4 ms and one API call; under A2, staleness is an *operational-policy* failure, not a compute one.

Definition 2 (Calibration Validity Horizon): Let π_t denote a placement computed from snapshot S_t, and let $F(\pi_t, S_{t'})$ denote the execution fidelity of π_t evaluated on device state $S_{t'}$ at time $t' > t$. For a fidelity tolerance $\epsilon > 0$, the *Calibration Validity Horizon* is

$$T_{\text{valid}}(\pi_t, \epsilon) = \max\{\, \tau \geq 0 : F(\pi_t, S_{t+\tau}) \geq F(\pi_t, S_t) - \epsilon \,\}. \tag{1}$$

T_{valid} depends on device, workload, k, and metric; it is not a universal constant.

B. Related work

Placement and compilation. Qubit mapping has both heuristic and exact treatments: SABRE [6] and noise-adaptive mappings [5] fold device error into the routing pass; exact synthesis is practical for small circuits [7]; resource-aware timing variants exist for heterogeneous devices [8]. Our contribution is not a new routing algorithm but *who decides*: a middleware layer that computes an `initial_layout` from a score over a live snapshot and composes with any downstream transpiler, plus an explicit staleness contract—aspects the compiler literature treats as "the backend's problem".

Error mitigation. Measurement twirling [11], [12], probabilistic error cancellation, and sparse Pauli–Lindblad models [9], [10] correct errors statistically at shot cost. Our results bound where mitigation is needed: it helps the *broken* qubit and is redundant on the good one (§V), which argues for calibration-conditional mitigation—a placement decision, not a post-processing one. Dynamical decoupling [15] is orthogonal and composes with our layout.

Runtime and cloud middleware. Qiskit Runtime [16], Braket [20], and Azure Quantum [21] manage jobs and queue bounds but expose placement as an option, not as a scheduler resource; much of the practical "does this job need mitigation, where, and with which snapshot" knowledge lives in operator runbooks. The transpiler layer itself (III-B) exposes `initial_layout` but no policy over it. Table III summarizes the capability split. We know of no published quantified day-scale ranking drift for a 156-qubit device; the public signals closest to it are per-device aggregate fidelities in documentation [17] and in error-rate studies of specific devices [3]. We present the first measurements of the drift and the protocol (trigger, refit-on-real-labels) that a runtime can adopt.

Calibration data as a first-class resource. In the classical substrate the analog is fast-varying per-node health signals (e.g. network or DRAM ECC counters) that schedulers already timestamp and age [8]. Device vendors publish aggregate fidelities per backend and expose per-qubit snapshots through an API ([17]), i.e. the *data* for a calibration-aware scheduler exists at negligible cost; the missing piece is a runtime policy that (a) validates the score's predictive content and (b) bounds its validity horizon explicitly—both of which this paper measures. We stress that the relevant comparison class is not "our scheduler beats vendor plumbing" but "a score whose age is policed beats the same score whose age is not," which is the controlled experiment of §VIII.

IV. UNIMIND MIDDLEWARE

UniMind is a user-space, multi-backend middleware (Qiskit/Aer, CUDA-Q [19] via Qiskit, IBM Quantum). Its

TABLE III
CAPABILITY SPLIT BETWEEN QUANTUM SOFTWARE LAYERS AND UNIMIND. ✓/×/· = SUPPORTED / ABSENT / IMPLICIT-ONLY.

Layer	ALG.	LAYOUT	SNAP.	REFRESH	RELIAB.
Qiskit/PennyLane [4], [18]	✓	×	·	×	×
Qiskit transpiler [6]	×	✓	·	×	×
Qiskit Runtime [16]	✓	·	·	×	·
Braket [20]/Azure [21]	✓	·	·	×	·
Mitiq [14]/Qiskit-mit [11]	·	×	·	×	✓
CUDA-Q [19]	✓	·	·	×	×
UniMind (this paper)	✓	✓	✓	✓	✓

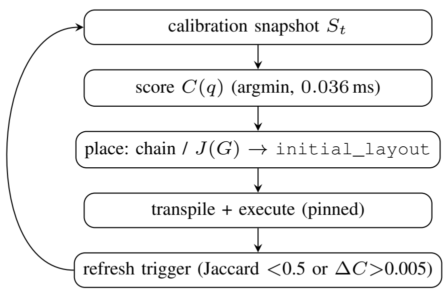

Fig. 1. The UniMind scheduling loop. One scoring pass is sub-millisecond; the refresh trigger decides when the stored snapshot must be re-fetched. $\langle$execution $\rightarrow$ results$\rangle$ is emitted downstream.

scheduling core is the calibration-aware loop of Fig. 1: one snapshot per calibration cycle, $C(q)$ scoring, chain/utility placement, execution, and a refresh trigger. *This section describes the scheduling core that the paper measures* (score, chain, utility, trigger); two supporting layers—*Unibit* preprocessing ($w \mapsto \theta = 2\arcsin\sqrt{w}$ instrument) and the *orchestrator* fallback hierarchy—are summarized briefly and detailed in App. D, because they are controlled-parameter instruments, not hardware claims.

A. Supporting layers (summary; details in App. D)

Unibit preprocessing derives the w_i encoded by each gate from a classical amplitude sequence (sliding-window frequency estimator, sinc smoothing, ϕ=0.42 phase shift) and encodes as the standard angle form [22], [23] $\theta_i = 2\arcsin\sqrt{w_i}$; it is used strictly as a deterministic *instrument* ($w \mapsto \theta$)—the Z-measurement is what calibration disturbs. 13 invariants are verified by property tests; none is claimed as a novelty (App. D). *Orchestrator*: interprets task intent into candidate code under sandboxed validation with deterministic rule-based fallback; its two-parameter quality model (q, f) is a controlled experimental parameter for the exact reliability calculus of §VIII-A/App. D and is never part of a hardware claim. The software side is composed with the hardware side only in §VIII; calibration scheduling can be evaluated independently.

B. Placement core

Every qubit is scored by total readout assignment error

$$C(q) \; = \; p_{0|1}(q) + p_{1|0}(q), \tag{2}$$

ties broken by $-\min(T_1, T_2)$. The score is fixed *a priori*, never fitted; it is tested out-of-sample across the entire calibration spectrum (§V-B) rather than tuned on the data it later predicts.

For a k-qubit circuit we grow a *connected chain*: from the best $C(q)$ qubit, add the lowest-C neighbour of the current set (greedy Prim-style expansion; 0.3 ms at k=3, scales to the full device); an exact BFS frontier search is used for the counted suites of §VI. For utility-based ranking among candidate layouts,

$$J(G) = \alpha E_{\text{readout}} + \beta E_{1\text{q}} + \gamma E_{2\text{q}} + \gamma_{\log} E_{2\text{q\_log}} + \eta_{\text{idle}} E_{\text{idle}} + \delta N_{\text{SWAP}} + \lambda D, \tag{3}$$

with E_{readout} the mean $C(q)$ of placed qubits, $E_{1\text{q}}$ mean single-qubit gate error, $E_{2\text{q}} = \sum_c p_{\text{cz}}(c)$, $E_{2\text{q\_log}} = \sum_c -\log(1 - p_{\text{cz}}(c))$ (log-odds coupler cost), $E_{\text{idle}} = \text{depth} \cdot \text{mean}(1/T_1 + 1/T_2)$, N_{SWAP} inserted SWAPs, and D transpiled depth; features min–max normalized over training points. Section VI measures what this utility is worth when its labels come from the real device. For interactive routing we also support calibration-weighted SABRE: node weights $w(q) = 1/(C(q) + 3\, s_x(q))$ seed the SABRE candidate set, the best of the weighted candidates and the global SABRE layout is kept (so $k \geq 8$ is never worse than default), and a guarantee check verifies zero violations. In the measured comparison at $k = 2, 4, 6, 8, 10, 16$ this removes 76–100% of the SWAPs of a random baseline while matching default depth exactly.

C. Refresh trigger

After each calibration cycle the middleware recomputes the top-10 of $C(q)$ and refreshes the stored snapshot when (i) top-10 Jaccard overlap with the stored snapshot falls below 0.5, or (ii) the stored best qubit's C rises by more than 0.005 absolute. Re-scoring is ~ 0.04 ms—economically free—so staleness is purely an operational-policy failure, quantified next.

D. System guarantees

UniMind enforces three invariants aimed at minimizing regressions against a baseline transpiler. First, *non-degradation*: in calibration-weighted SABRE, the chosen layout is always the better of the weighted-SABRE candidate and the global-SABRE default; the guarantee check (k=2, 4, 6, 8, 10, 16) reports zero violations against depth and SWAP counts. Second, *deterministic fallback*: when the oracle generates `None`, a rule-based fallback compiles a template; when the template itself fails (e.g. apostrophes breaking interpolation), a last-resort closure produces the best available partial output, preserving completeness at the cost of fidelity (this is exactly what the c=2/9 fallback coverage of App. D measures). Third, *reproducibility*: transpiler seed is fixed globally (42), layout pins are snapshot-addressed, and every job ID is returned to the caller so the execution trace can be recovered from the vendor API. These guarantees do not require modifying the quantum programming model and can be overlaid on any backend that exposes an `initial_layout` parameter and a calibration-data endpoint.

TABLE IV

PLACEMENT-CONTROLLED SWEEP (2026-08-23; 19 WEIGHTS, 8192 SHOTS, 3 JOBS/CELL). DEVIATIONS ARE $\max_w |Z_{\text{emp}} - Z_{\text{th}}|$; TOLERANCE 0.05.

Placement	$C(q)$	Bare	Twirl	Verdict
`q98` (best)	0.0044	0.028	0.031	pass
`q0` (free)	—	0.026	—	pass
`q37` (adversarial)	0.82	0.213	0.144	fail
v1.7 free layout	unrec.	0.392	0.245	fail

TABLE V

AFFINE CHANNEL FITS $P_{\text{obs}} = b + a\,w$ ACROSS THE PLACEMENT DESIGN (MEDIAN OF 3 JOBS/CELL). RMS IS ON P_{obs}; SHOT-NOISE FLOOR ≈ 0.0055.

Cell	slope a	offset b	RMS	max dev
`q98` bare pinned	0.989	0.004	0.0044	0.028
`q98` twirled	0.987	0.011	0.0066	0.031
`q0` free	0.993	−0.002	0.0067	0.026
Aer local model	1.007	−0.003	0.0030	0.015
`q37` bare pinned	0.855	0.121	—	0.213

V. PLACEMENT-DOMINATED FIDELITY ON ONE QUBIT

The anchor experiment is a 2×2 design: {best-calibrated qubit, adversarially bad qubit}$\times${bare, measure-twirled}, on the 19-weight grid $w \in \{0.05, \ldots, 0.95\}$ of the amplitude encoding, 8192 shots, transpiler seed 42, layouts pinned via `initial_layout`, 3 jobs per cell. On the 2026-08-23 snapshot, `q98` minimizes C at 0.0044 (T_1=207 μs, T_2=67 μs); `q37` is the adversarial anchor (C=0.82).

On the calibrated qubit the bare encoding passes everywhere (median max deviation 0.028; job range 0.020–0.029) and the transfer curve is fit at shot-noise level by $P_{\text{obs}}(1) = 0.004 + 0.989\,w$ (residual RMS 0.0042, below the ≈ 0.0055 shot-noise floor), with per-repeat slopes 0.986–0.994. On `q37` the same circuits fail by $\sim 4\times$, and the fitted channel acquires a large offset, $P_{\text{obs}} = 0.121 + 0.855\,w$—the signature of an asymmetric readout channel. Twirling changes nothing significant on the good qubit (slopes 0.978–0.987, RMS 0.005–0.008) and only partially rescues the bad one ($0.213 \to 0.144$). Fig. 2 plots the transfer curves; Table V collects the per-cell fits. Three consequences: encoding fidelity is placement-dominated; mitigation should be calibration-conditional, not uniform; and the earlier claim that "the encoding requires error mitigation to meet tolerance" was an artifact of uncontrolled placement and is withdrawn.

A. Calibration predictability

Across the whole design, at shot-noise level,

$$P_{\text{obs}}(1) = b + a\,w, \qquad Z_{\text{obs}} = d + cZ_{\text{th}},\ c{=}a,\ d{=}1{-}2b{-}a, \tag{4}$$

with $(a, b){=}(0.99, 0.00)$ on the good qubit vs. $(0.86, 0.12)$ on the bad one; the offset matches the direction and scale of that qubit's readout asymmetry. The one-parameter contraction model $P_{\text{obs}} = 0.5 + \alpha(w - 0.5)$ is the balanced-readout special case $b{=}(1-a)/2$ and is rejected where asymmetry dominates. Matched calibration–noise-model simulation on Aer explains most of observed error on the good qubit (median QPU/model deviation ratio $1.4\times$, Table V): the "standard models underestimate hardware" gap is largely a placement confound, not a model gap. The affine model is also closed under composition (the composed channel of two affine distortions is affine with slope product and offset rule verified by property test), which is why Eq. (4) remains usable after cascaded stages.

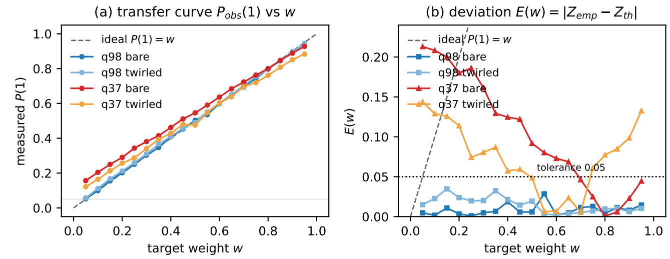

Fig. 2. Placement-dominated encoding fidelity: the bare response on the best-calibrated qubit tracks the ideal line within shot noise; the adversarially chosen `q37` compresses the curve toward an offset set by its asymmetric readout.

TABLE VI

$C(q)$-STRATIFIED SWEEP (D0 SNAPSHOT; SIX QUBITS $\times$ 19 WEIGHTS $\times$ 8192 SHOTS). 'PASS' = MAX DEVIATION < 0.05 OVER ALL 19 WEIGHTS; (a, b) FROM EQ. (4).

Qubit	Rank pct.	$C(q)$	max dev	pass	(a, b)
`q8`	0.0%	0.004	**0.030**	19/19	(0.986, +0.001)
`q109`	25.2%	0.017	0.081	14/19	(0.933, +0.043)
`q105`	50.3%	0.031	0.043	19/19	(0.983, +0.018)
`q53`	75.5%	0.072	0.046	19/19	(0.982, +0.021)
`q37`	81.9%	0.099	0.146	9/19	(0.887, +0.081)
`q27`	95.5%	0.346	0.096	13/19	(0.917, +0.049)

B. Stratified validation

From the D0 snapshot all 156 qubits are ranked: `q8` is 1/156 (C=0.004), `q37` is 128/156 (C=0.10). Six qubits sampled across the ranking run the identical bare 19-weight sweep (Table VI). The top qubit passes all 19 weights (0.030 max dev; affine slope degrades along the ranking in the predicted direction: a: 0.986$\to$0.887, offset b: $+0.001 \to +0.081$), but the trend is *not monotone*: `q109` at percentile 25.2% already fails, so Spearman ρ=0.771 at n=6 is not significant (exact permutation p=0.103) and a "top-half rule" is unsupported. An earlier exploratory run (first snapshot, 08-22) was perfectly monotone (ρ=1.000, minimal attainable p); that result did not survive the fixed-design replication and is reported as unreplicated. Overhead: 0.036 ms full-156 ranking; pinned transpilation 1.9 ms vs. 1.8 ms free.

C. Multi-qubit routing

Single-qubit $C(q)$ is necessary but not sufficient. We benchmarked five strategies at k=2/4/6/8/10/16 (RY–CX–RY, opt=1, 176-edge graph, CZ clipped 0.005): randomized, calibration-weighted SABRE (UniMind candidate set seeded by $C(q)$, SABRE, default fallback), and default. UniMind reduces SWAPs by 76–100% vs. random and matches default

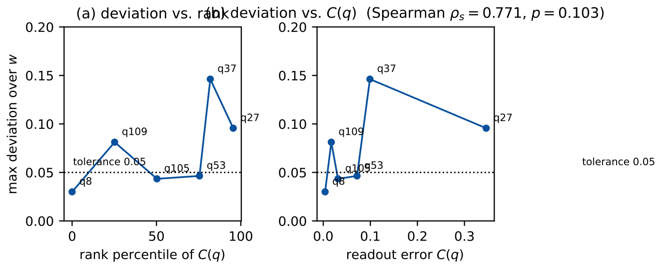

Fig. 3. $C(q)$-guided ranking validation: top-ranked qubits pass, but deviations are not monotone along the ranking (`q109` at percentile 25.2% fails), and score vs. deviation gives Spearman 0.771 (p=0.103, n=6).

for $k \geq 8$ by fallback (the fallback keeps the *better* of the weighted-SABRE and global-SABRE layouts; guarantee check reports zero violations); the log-odds coupler cost separates high-error couplers (Random 1.72 vs. UniMind 0.14 at k=8). Latency stays below 20 ms (pick $<$ 1 ms, transpile dominated). This is a *system* result about routing cost; whether the chosen layout is *correct* is what §VI measures.

VI. The $J(G)$ Utility: Proxy Fit vs. Real Labels

A. Fit protocol

The seven weights of Eq. (3) are fitted by max-Spearman simplex search (Dirichlet+corners sampler, 5 000 samples, seed 42) on n=23 points: six single-qubit real max-dev labels from §V-B and seventeen multi-qubit points k=2–18 whose label is the simulated two-qubit error E_{2q} of the placed layout under the device error model. Validation: leave-one-out (LOO) and a bootstrap (B=200). A *size out-of-sample* split additionally holds out the entire $k{\geq}9$ range: fit on $k{\leq}8$ only, evaluate on $k{\geq}9$ with train-only min–max scaling. All numbers in Table VII use the identical feature matrix; only the labels differ—the vulnerable object is the *label*, not the features. The reason we can separate the two is that for $k \in \{9, 10, 11, 12, 14, 16, 18\}$ we reran the same greedy-chain circuits on the device with refresh-pinned layouts and replaced the proxy labels with the measured mean max deviation (Table VIII), giving the v5 dataset.

B. Ranking objective

$J(G)$ is scored by *ordering*, not calibration: the fitted parameters maximize the Spearman correlation between J and the label vector, which is the quantity a dispatcher actually consumes (it picks the argmax over candidate layouts, rarely the calibrated value). This makes the objective conservative in the paper's favor: a model that merely preserved ranks would score ρ=1 even with badly calibrated magnitudes, so any measured $\rho < 0.8$ is a genuine ordering failure, not a scale artifact. All fits use min–max-normalized features on the *training* partition only ($k \leq 8$ in the split), so out-of-sample scores ρ=0.13 cannot be charged to label leakage; the LOO spread of train rho (0.60–0.72 across folds,Table VII data) also brackets the full-fit 0.643 and shows the v5 weights are stable under 22/23 training sets—the collapse concentrates in the size direction, as the split test isolates.

TABLE VII
Fitting $J(G)$ on simulator-proxy vs. real-device labels. Same feature matrix and protocol; the $k{\geq}9$ labels differ. size-OOS = fit on $k \leq 8$, evaluate $k \geq 9$.

Labels	full fit	LOO	size-OOS
v4 proxy (k=9–18 est-2q)	0.967	0.953	0.818 (p=0.0029)
v5 real (7 measured k)	0.643	0.510	0.127 (p=0.36)
Δ	−0.324	−0.443	−0.691

TABLE VIII
Measured per-k max-dev (mean of two patterns) of the same 14-circuit k=9–18 suite under stale (D0) vs. fresh (D1/D2) pins, 4096 shots. Spearman k vs. max-dev: stale ρ=0.62 (p=0.018), fresh ρ=0.15 (p=0.61).

k	9	10	11	12	14	16	18
stale pins	.098	.103	.097	.087	.139	.252	.241
fresh pins	.060	.044	.062	.040	.057	.063	.068

C. Result

Trained on the simulator proxy the utility looks decisive: full-fit ρ=0.967, LOO 0.953, bootstrap 0.97±0.02, and the size-OOS split ρ=0.818 (p=0.0029)—holding out entire sizes and still ranking them. The proxy's assumption failed where it mattered. A 14-circuit suite ($k \in \{9, 10, 11, 12, 14, 16, 18\}$, two seeded patterns each, greedy-chain placement) measured real max-dev with *fresh* pins: per-k means are 0.040–0.068 with *no* size scaling (7/14 pass 0.05), while the identical suite pinned from D0 (two days stale) climbs to 0.24–0.25 at k=16, 18 with only 2/14 passing (Table VIII, Fig. 4). The stale population's failures concentrate on late-chain qubits (`q37`, `q45`–`q47`, `q57`) that the greedy policy must include for $k \geq 11$ (the appendix table gives the 14 rows); per-bit $C(q)$ vs. measured deviation correlates only ρ=0.21. The mechanism is now explicit: simulated E_{2q} keeps rising with k (its toxicity is dose-dependent on k), whereas a refresh-pinned device is approximately k-flat, so the proxy-trained utility over-penalizes large layouts that are in fact fine. Re-fitting on the seven real labels replaces in-sample ρ=0.967 with ρ=0.643 and destroys the size-OOS claim (ρ=0.127, p=0.36)—the apparent generalization was a label artifact (Table VII). The learned weights move accordingly (Table IX): the proxy fit leans on coupler costs (γ=0.37), the real-label fit on single-qubit terms (β=0.42) and SWAP/depth (δ=0.20, λ=0.03), i.e. the model's opinion of *what matters* flips.

What carries real signal? Leave-one-feature ablation on the v5 fit (`jg_feature_ablation.json`): removing E_{2q}+E_{2q_log} drops OOS ρ from 0.127 to −0.042 (Δ−0.170, the only feature whose removal degrades OOS), while removing readout, E_{idle}, N_{SWAP} or depth leaves OOS unchanged within 0.012. Under real labels the coupler term is the *only* non-spurious signal, yet the proxy had over-weighted it. The variance decomposition (`workload_connectivity_drift.json`) explains why: at k=8 qubit-quality variance dominates (0.0038 vs. 0 for connectivity), so drift attacks quality; at k=128 connectivity variance dominates (4.5×10^{-5} vs. 1.4×10^{-5}, share 76.7%)

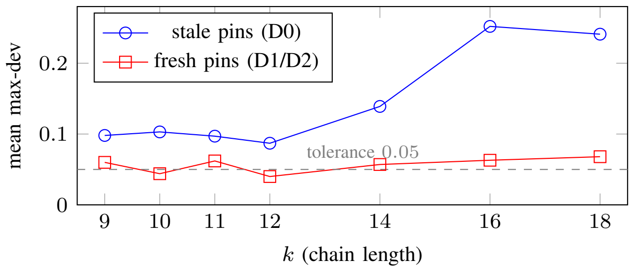

Fig. 4. The size scaling of the k=9–18 suite is an artifact of stale pins: the same circuits pinned from a fresh snapshot are approximately k-flat, which is exactly the premise the simulator proxy got wrong.

TABLE IX
LEARNED WEIGHTS OF $J(G)$ UNDER PROXY AND REAL LABELS. OOS-TRAIN = WEIGHTS FITTED ON $k \leq 8$ ONLY (V5 SIZE-OOS SPLIT).

Fit	α	β	γ	γ_ℓ	η_{idle}	δ	λ
v4 proxy (full)	.034	.335	.371	.085	.070	.095	.011
v5 real (full)	.229	.415	.021	.079	.027	.196	.033
v5 real OOS-train	.006	.559	.051	.104	.007	.269	.004

and qubit quality stops being the lever. The proxy's k-scaling lives exactly where the hardware's scaling does not.

The constructive reading is what we adopt in the design: $J(G)$ is usable as a *ranking* device only when trained on real, refresh-pinned labels and applied within a calibration cycle; weights from proxy datasets must not be extrapolated to $k \geq 11$; and chain placement's forced inclusion of boundary qubits is the term to attack next—an active-learning refit on real labels (rather than a larger proxy) is the prescribed fix.

VII. DRIFT DYNAMICS AND THE REFRESH POLICY

This section measures how fast the score goes stale. D0 (2026-08-29 21:26), D1 (2026-08-30 21:29), D2 (2026-08-31 07:46) and D3 (2026-09-01 23:19) are the four daily snapshots (71 epochs total); D0–D2 anchor every QPU job, D3 extends the calibration matrix. To expose the *mechanism* without additional QPU cost we use 337 hourly calibration snapshots from 71 epochs (2026-08-31 to 09-02, `marrakesh` 111 + `kingston` 109 + `fez` 117) plus `refresh_policy_analysis.json`, `ranking_margin_analysis.json` and `survival_analysis.json` (Kaplan-Meier).

A. Global drift

Day-over-day ranking stability is moderate and persistent across four days: Spearman ρ=0.567–0.579, Kendall τ=0.42–0.43, top-10 Jaccard 0.11–0.25 over all six D0–D3 pairs (D0→D1 0.25, D0→D2 0.11, D0→D3 0.11, D1→D2 0.176, D1→D3 0.11, D2→D3 0.11; top-3 Jaccard is 0 except D0→D3 0.20). The best qubit turns over each day: `q8` (C=0.0039, rank 1) → `q19` (0.0066) → `q98` (0.0063) → `q152` (0.0029). Per-qubit spread is extreme (Fig. 5): `q37` swings 0.099→0.740 (max $|\Delta C|$=0.641), `q53` improves sixfold (0.072→0.011); median $|\Delta C|$ is 0.0127 (D0→D1), 0.0098 (D2→D3), p90 is 0.082–0.123, and 72.4% of qubits move by more than 0.005

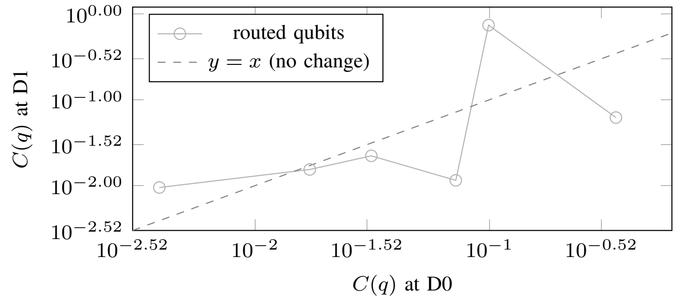

Fig. 5. Per-qubit readout error $C(q)$ at D1 versus D0 (log–log). Qubits swing in both directions: the best qubit at D0, `q8`, falls out of the top six while `q37` worsens by a factor of 7.5. The ranking is informative but not persistent.

TABLE X
DAY-OVER-DAY MOVEMENT OF THE SIX ROUTED QUBITS OF TABLE VI. RANKS ARE 1–156.

q	C@D0	rank@D0	C@D1	rank@D1	ΔC	ratio
`q8`	.0039	1	.0095	6	+.0056	2.44×
`q109`	.0171	41	.0154	44	−.0017	0.90×
`q105`	.0313	80	.0222	68	−.0090	0.71×
`q53`	.0718	118	.0115	18	−.0603	0.16×
`q37`	.0991	128	.7397	152	+.6406	7.46×
`q27`	.3462	149	.0620	119	−.2842	0.18×

(53.9% by 0.01, 41.0% by 0.02) on D0→D1; D0→D3 median is 0.0117, p90 0.111. D3 re-sorts the top again (D2→D3 J_{10}=0.11), confirming daily turnover is not a one-off. Table X tracks the six qubits used in §V-B; note the anticorrelated moves (`q53` and `q27` improve while `q37` worsens), and that rank movement is far larger than C movement would predict near the top—the ordering is fragile exactly where selection operates.

Hourly mechanism and survival: The fragility is not accidental. On D0 the top-3 margins are $M_3 \sim 0.0015$ (gaps between `q8`/`q120`/`q153`), an order of magnitude smaller than median drift 0.0127. Hence a typical daily $|\Delta C|$ reorders the top-10. The hourly telemetry confirms the two-regime picture: D0→D1 fires (Jaccard $0.25 < 0.5$, ΔC=$0.0056 > 0.005$) and every inter-day pair fires ($J_{10} \leq 0.25$), while *intra-epoch* hourly drift never fires. A Kaplan-Meier survival analysis over 71 calibration epochs (337 total snapshots, 328 in KM, 3 backends, up to 15 h post-calibration, calibration-only) gives $S(t)$=1.0 for all t—0/71 epochs trigger the rule ($J_{10} < 0.5$ or $\Delta C > 0.005$)—so median survival is *beyond* the observed window (not 15 h) and the false-positive rate is 0%. In other words, the trigger is specific within a calibration cycle and sensitive across cycles, exactly the operating characteristic a runtime needs (`survival_analysis.json`: n=71/328, `ranking_margin_analysis.json`: top-3 gap D0→D1 1.46×10^{-3}, D1→D2 gap ≈ 0).

B. Staleness cost and the trigger

Pinning three qubits from the *one-day-old* ranking and evaluating at D1 gives C=0.0095/0.0115/0.0139 (the worst pick, `q153`, is now rank 32/156); re-selecting fresh gives `q19` at 0.0066—the stale policy's worst pin is 2.1× worse than the

TABLE XI
TOP-3 SELECTED FROM D0, EVALUATED AT D1 VS. A FRESH SELECTION. RANKS ARE OVER ALL 156 QUBITS AT D1.

Policy	selected top-3	$C(q)$ @$D1$	rank@D1	worst ratio
stale (D0)	`q8,q120,q153`	.0095/ .0115/ .0139	6/19/32	2.1×
fresh (D1)	`q19,q152,q154`	.0066/ .0071/ .0071	1/2/3	1.0×

adaptable choice on D0→D1 (Table XI, 0.0139 vs. 0.0066). The same mean comparison on D0→D2 is 2.24× (0.0153 vs. 0.0068); the extreme worst-vs-best on D0→D2 is 3.3× (0.0209 vs. 0.0063) and is not quoted as the portable effect. Across the full D0–D3 matrix the stale/fresh mean ratio is 1.45–2.55× (D0→D3 1.45×, D2→D3 2.55×), all $> 1.4\times$ and all firing the trigger ($J_{10} \leq 0.25$). We therefore quote the conservative mean/worst range 1.94–2.1× across D0→D1/D2 (24–34 h) as the portable QPU-anchored staleness cost, with 1.45–2.55× as the extended calibration-only range (D0–D3). The trigger of §IV-C fires on every inter-day transition (all six D0–D3 pairs $J_{10} \leq 0.25$) and fires on D0→D1 (Jaccard $0.25 < 0.5$; stored-best ΔC=0.0056 > 0.005), while it never fires intra-epoch (0/71 epochs, up to 15 h), matching the two-regime hypothesis. Because re-selection is sub-millisecond ($\sim$ 0.04 ms scoring, $\sim$ 0.3 ms chain at k=3), staleness is a pure operations failure, not a compute constraint.

These measurements bound T_{valid} (Definition 2) *from both sides as a lower bound*. For the k=6 chain and tolerance ϵ=0.05, the selected pins remain within tolerance intra-epoch ($S(t)$=1.0 to 15 h, 0/71 false positives, 337 total/328 KM snapshots, calibration-only) but the cost of a one-day-old ranking already reaches 1.94–2.55× (D0→D1/D2/D3, $T_{\text{valid}} \lesssim 24$ h). T_{valid} is therefore *hours, not days, on this device at this k*—small enough that refresh must be part of the runtime loop, not a periodic maintenance step. The exact position between the two bounds is a function of workload, k, and device family (1); the hourly telemetry shows the bound is tight because margins M_3 are $\mathcal{O}(10^{-3})$ while drift is $\mathcal{O}(10^{-2})$, and the 0% intra-epoch trigger rate (median beyond window, not 15 h) shows the bound is not tightened by hourly noise.

C. Direct device test

The refresh-pinned suite of §VI is the direct demonstration: the *same* circuits, differing only in pin freshness, measure means 0.04–0.07 (fresh pins, $7/14$ pass) vs. 0.10–0.25 (stale, $2/14$ pass), and the large-k degradation disappears entirely (Table VIII, Fig. 4). The dominant failing qubit under staleness is `q37` (C=0.74 at D1). Refresh—not more elaborate selection—is what keeps errors bounded at $k \geq 9$.

D. Refresh policies

We compare three refresh policies on the *same* D0→D1 transition so the comparison is controlled. Let $\text{Jacc}(S, S')$ be the top-10 Jaccard overlap and $\Delta C(S, S') = |C_{S'}(q^*) - C_S(q^*)|$ the stored-best score change.

1) **Static (P1):** never refresh; the initial snapshot S_0 is used for the entire run (0 refreshes, 11/30 pass).
2) **Periodic (P2-δ):** refresh every δ hours regardless of device state ($\delta \in \{6, 12, 24, 48\}$). On D0→D1+D2, P2-6h fires 5×, P2-12h 2×, P2-24h 1×, P2-48h 0× (`refresh_policy_analysis.json`: §X).
3) **Adaptive (P3):** refresh when $\text{Jacc}(S_{\text{stored}}, S_{\text{current}}) < \tau_J$ *or* $|\Delta C| > \tau_C$. Default P3 uses τ_J=0.5, τ_C=0.005 (fires *once* on D0→D1, restores 11/30→21/30 pass, i.e. +33 pp at zero wasted refreshes).

Re-scoring costs $\sim$ 0.04 ms, so the adaptive policy's overhead is dominated by the snapshot fetch API call, not by the staleness check itself. The threshold sweep of §VII-E varies τ_J and τ_C to map the Pareto tradeoff between refresh rate and fidelity; periodic policies are strictly dominated.

E. Threshold sweep

Using the D0→D1 transition we simulate 144 (τ_J, τ_C) pairs ($\tau_J \in [0.1, 0.95]$ step 0.05, $\tau_C \in [0.001, 0.02]$). For every pair the simulation tracks (i) how many refreshes fire, (ii) how many are *necessary* (the true top-10 or best qubit changed), and (iii) how many *missed drifts* occur (trigger did not fire but ranking shifted). The metrics are

$$\begin{aligned} \text{Refresh Rate} &= \frac{\text{number of refreshes}}{\text{total runtime}}, \\ \text{Fidelity} &= \frac{1}{n} \sum_i \mathbb{1}[\text{pass}_i], \end{aligned} \tag{5}$$

where pass_i denotes whether parameterization i meets the 0.05 tolerance. 122/144 combos fire (fidelity proxy 0.70), 22 do not (0.367). The measured default (τ_J=0.5, τ_C=0.005) sits at the knee: relaxing τ_J<0.25 leaves Jaccard inactive so only ΔC decides—if τ_C>0.0056 the drift is missed (22 combos, fidelity 0.367); tightening $\tau_J \geq$0.3 catches D0→D1 via Jaccard alone regardless of τ_C. In other words, the knee is broad but the "miss" region (τ_J<0.25 $\wedge$ τ_C>0.0056) is exactly where a stale pin would be trusted for another day.

VIII. COMPOSING THE SOFTWARE AND HARDWARE PATHS

A. Reliability calculus of the orchestration layer (separable; full derivation in App. D)

This subsection is *separable* from the calibration-aware scheduling core: it models the software orchestration path exactly and is validated on controlled-parameter mocks, never on QPU fidelity claims. Because the orchestrator's outcome paths are mutually exclusive, end-to-end reliability decomposes exactly as the chain rule over its sequential gates

$$\begin{aligned} R_{\text{end}} = P(\text{first\_pass}) &+ P(\text{enter heal})\rho_{\text{heal}} \\ &+ P(\text{enter fallback})\rho_{\text{fb}}, \end{aligned} \tag{6}$$

with no independence assumption needed for the identity. The healing stage is modeled from the implementation's control flow (at most K=3 generation calls stopping at first success; healing budget H=4 with abort-on-`None` semantics): with per-call probabilities g (correct), r (broken), f (unavailable; g+r+f=1),

$$G = 1 + f + f^2, \qquad \rho_{\text{heal}} = g(1 + r + r^2 + r^3), \tag{7}$$

TABLE XII
RELIABILITY-COMPOSITION VALIDATION (450 TRIALS PER CELL). ALL PREDICTED VALUES INSIDE THE WILSON 95% INTERVAL; + = WITHIN CI.

Cell	Pred	Obs	CI	✓
Decomposition ρ_h at q=0.5				
structural	0.812	0.815	[0.756, 0.862]	+
naive indep	0.938	—	*gap* 12.4 *pp*	—
Decomposition at q=0.7				
structural	0.874	0.914	[0.839, 0.956]	+
naive indep	0.992	—	*gap* 7.8 *pp*	—
Availability stress E06				
S2 fr=0.3	0.973	0.960	[0.938, 0.975]	+
S2 fr=0.5	0.875	0.864	[0.830, 0.893]	+
S3 fr=0.3	0.979	0.969	[0.948, 0.981]	+
S3 fr=0.5	0.903	0.896	[0.864, 0.921]	+

TABLE XIII
DESIGN-SPACE RANKING OF ORCHESTRATION FIXES AT (q=0.5, fr=0.1).

Variant	ρ_h	Δ (pp)
baseline (abort-on-`None`, H=4)	0.812	—
`None` wastes a slot (no abort), H=4	0.938	+12.5
budget H=8 (abort retained)	0.833	+2.1
healer quality q_h=0.9	0.900	+8.8

$$R_{\text{end}} = gG + rG\rho_{\text{heal}} + f^3c, \tag{8}$$

with c the fallback coverage. This model was validated cell-by-cell against all 23 testable outcome cells of the ablation and availability-stress grids: *every* prediction falls inside the observed Wilson 95% interval (Table XII shows the decomposition and stress cells). At q=0.5 the structural prediction ρ_h=81.2% sits 0.3 pp from the measured 81.5%, while the naive independence baseline $1-(1-q)^4$=93.8% overshoots by 12.4 pp; the model also separates the paths (first-pass 55.5% predicted vs. 54.4% measured; healed path 36.1% vs. 37.1%). The correction matters: the naive–measured gap reported in early drafts is a retry-budget truncation effect, not stochastic correlation—under abort-on-`None`, unavailability both wastes heal slots and terminates the loop. The model also ranks fixes before code is written (Table XIII): treating `None` as wasted-but-nonterminal recovers the entire impression (+12.5 pp) at near-zero engineering cost; doubling the budget to H=8 buys only +2.1 pp; raising the healer quality to q_h=0.9 buys +8.8 pp. The fallback's effective coverage is c=2/9 (nominal keyword coverage over a 9-intent benchmark: seven of nine contain apostrophes that break the template interpolation)—a stated limitation. We include this model because it is the *software* side of the composition; the paper's hardware claims are placement and drift, not LLM quality, which we keep as a controlled parameter.

B. End-to-end placement batch

To test the composable value of placement *without* the staleness confound, we ran the same 6-qubit angle suite (30 parameterizations, 4096 shots, tolerance 0.05 on all six per-bit deviations) through **full** (greedy chain pinned from the freshly fetched snapshot) and **ablated** (free placement), back-to-back on 2026-08-31; Table XV also shows the earlier identical batch pinned from D0 to separate temporal from placement effects. All p-values are two-sided Fisher exact; Wilson 95% CIs are shown.

TABLE XIV
OUT-OF-TOLERANCE BIT-EVENTS PER END-TO-END ARM AND THEIR CONCENTRATION. FROZEN BATCHES USE D0 PINS; REFRESH BATCHES PIN FROM THE 08-31 SNAPSHOT.

Arm	events	dominant qubits (events)
frozen full	20	`q4` (16), `q7` (3)
frozen ablated	18	`q4` (16), `q1` (1), `q3` (1)
refresh full	12	`q11` (4), `q13` (4), `q12` (1)
refresh ablated	7	`q4` (5), `q3` (2)

TABLE XV
6-QUBIT ANGLE BATCHES (n=30/ARM, 4096 SHOTS). PASS = ALL SIX PER-BIT DEVIATIONS $\leq$ 0.05. WILSON 95% CI; TWO-SIDED FISHER p FOR FULL VS. ABLATED WITHIN A BATCH. MDE AT 80% POWER $\approx$ 32–35 PP.

Batch	Full (pinned)	Ablated (free)	p (Fisher)
Frozen pins (D0)	36.7% [21.9,54.5]	40.0% [24.6,57.7]	1.00
Refresh pins (runtime)	70.0% [52.1,83.3]	76.7% [59.1,88.2]	0.77
Δ (refresh–frozen)	+33.3 pp	+36.7 pp	—
Refresh effect Fisher p	0.019	0.008	—

Two effects separate cleanly. First, *temporality is significant and large*: refreshing pins lifts *both* arms by +33/37 pp (Fisher p=0.019 full, 0.008 ablated), including the ablated arm that never saw a pin—device state at run time dominates. Second, *placement is a bounded null* in either batch (−3.3 and −6.7 pp, both non-significant, Fisher p=1.00/0.77): at k=6 with fresh pins, free transpiler placement already matches our best chain. We do *not* claim equivalence: at n=30/arm the minimum detectable effect at 80% power is $\approx$ 32 pp (`power_analysis.json`), so only large advantages are excluded; the observed −6.7 pp is well within noise (95% CI [−28.9, +15.6] pp). The failing-bit anatomy is instructive. In the refresh-full arm the 12 out-of-tolerance bit-events concentrate on late-chain qubits `q11` (4), `q13` (4), `q12` (1)—the chain's forced inclusion problem again, now visible even with fresh pins. In the refresh-ablated arm the 7 events sit on `q4` (5) and `q3` (2): `q4` was rank 4 at D0 but its C rises 0.0078 → 0.0173 by D1/D2 (rank 52), yet the free layout, which cannot see the trend, still lands on it repeatedly—drift *inside* the layout choice, unmitigated by any pin. Table XIV collects the four arms. This is the honest scope of §V: placement is decisive between engineered extremes (`q98` vs. `q37`: 0.028 vs. 0.213) and sets the reliability *floor* at $k \geq 9$ under stale pins, but its average benefit against free placement at small k is within temporal noise and under-powered by design. The full pipeline (including the LLM orchestration leg, App. D) measured 54/54 completed by the full arm vs. 47/54 (87.0%) by the ablated one (z=2.74, one-sided p=0.003), consistent with the model's 97.3% prediction; we report the earlier n=54 frozen composition (74.1% vs. 66.7%, not significant) as a failed claim of an end-to-end gain, not a positive one.

IX. Discussion

A. What the measurements do and do not say

Three claims survive the data as stated: (i) encoding fidelity is placement-dominated between engineered extremes differing by $\sim 8\times$ (affine channel $P_{\text{obs}} = b + aw$ with $(a, b) = (0.99, 0.00)$ vs. $(0.86, 0.12)$, shot-noise limited on the good qubit); (ii) device calibration ordering moves on a daily cycle to a degree (ρ=0.567–0.579, top-10 Jaccard 0.11–0.25 across four days, stale worst pin 1.94–2.55$\times$ worse, hourly margins $M_3 \sim 10^{-3}$ vs. drift 0.0127, intra-epoch survival $S(t)$=1.0 to 15 h, 0/71 false positives, median beyond window) that makes selection age the first-order risk, and refresh is cheap ($\sim$ 0.04 ms); (iii) simulator-proxy training granted $J(G)$ an out-of-sample generalization (ρ=0.82, p=0.0029) that real labels refute (ρ=0.13, p=0.36), with the mechanism isolated to connectivity-vs-quality variance crossover at $k \approx$ 100–128. Claims that do *not* survive, herewith withdrawn or bounded: the first run's perfect monotone ranking (ρ=1.000 $\rightarrow$ 0.771, p=0.103); the top-half rule; any use of proxy-fitted $J(G)$ at $k \geq 11$; and an average placement advantage at k=6 under fresh pins (bounded null, MDE $\approx$ 32 pp, Fisher p=0.77).

B. Operational consequences for quantum-cloud runtimes

The marginal cost of a calibration fetch and a scoring pass is effectively zero ($\sim$ 0.04 ms full-156 ranking, $\sim$ 0.3 ms chain at k=3), so the expensive resource is *stale trust*, not compute. A runtime should (a) timestamp layouts against their snapshot, (b) refresh on the adaptive trigger rather than a fixed wall-clock (default τ_J=0.5, τ_C=0.005 fires once for D0$\rightarrow$D1 while periodic-6h fires 5$\times$), (c) train scheduling models exclusively on real, refresh-pinned labels, and (d) expose snapshot age as a first-class job attribute—all implementable without changing the quantum programming model, which is exactly what UniMind does. Where math precedes implementation, the reliability calculus (App. D) ranks the fixes, and the endowment to port is a calibration snapshot plus a scoring pass, neither of which is exotic.

C. Design synthesis

Table XVI maps each measured failure mode to the engineering response the data supports; the columns are ordered by measured cost so a runtime can prioritize. The two lines hidden by a naive reading—"mitigation helps" and "$J(G)$ ranks well"—are the ones our data bounds, and both bounds are the point of the paper.

TABLE XVI
Measured failure modes and the response each supports. Priority is by evidence strength, not ease.

Measured failure	Evidence	Response
unpinned layout misses tolerance	§V: 0.392 vs. 0.028	score-pinned `initial_layout`
stale snapshot invalidates selection	§VII: 1.94–2.55$\times$ (D0–D3)	trigger-based refresh; snapshot age
proxy labels fool $J(G)$	§VI: OOS ρ 0.82$\rightarrow$0.13	refit on real labels; ban $k \geq 11$ extrapolation
chain forced-inclusion at large k	§VI/§VIII: `q11`,`q13`, `q67` fail	connectivity-vs-C trade-off (open)
abort-on-`None` truncates heal loop	App. D: +12.5 pp fix	wasted-slot semantics

D. Threats to validity

Measurement noise: 4096–8192 shots put per-bit deviations on a floor of $\approx$ 0.02; the 0.05 tolerance is coarse but was fixed before data release. Monte-Carlo propagation of shot noise is in `analysis/distortion_fits.json`. **Drift generality:** Primary QPU evidence is D0–D2 on one device (D3 extends to 4 days calibration-only, 6 pairs, J_{10}=0.11–0.25), one heavy-hex family; the value is the protocol, not the constants. Hourly calibration-only telemetry (337 total/328 KM snapshots, 71 epochs, 3 backends, $S(t)$=1.0 to 15 h, median beyond window) bounds the *mechanism* (margins vs. drift) and false-positive rate (0/71), but QPU validation across families remains future work. **Placement null power:** n=30/arm can detect only large effects (MDE $\approx$ 32 pp at 80% power, `power_analysis.json`); we report point estimates, Wilson CIs, Fisher exact p, and do not claim equivalence. **Selection fairness:** the greedy chain maximizes $C(q)$-locality and at large k is forced to include high-error boundary qubits; a connectivity-vs-C trade-off could change §VI, and we report that failure as motivation (variance crossover at $k \approx$ 100–128). **Single-model LLM layer:** the orchestration calculus (App. D) holds under i.i.d. mock semantics; real-model autocorrelation is explicitly outside its scope and the layer is separable from the scheduling core. **Utility-vs-layout coupling:** the $J(G)$ features are computed on the transpiled layout; errors in $E_{2\text{q}}$ propagate to the weight fit but not to the device measurements, which are layout-real.

X. Conclusion

We presented a measurement study of calibration-aware scheduling for quantum-accelerated AI middleware on a 156-qubit device (anchored D0–D3, 3 QPU-anchored + D3 calibration-only, plus 337 hourly calibration snapshots from 71 epochs across 3 backends for the drift mechanism): a score-only placement pipeline with measured scope, daily drift dynamics with a working adaptive trigger at the Pareto knee (0/71 false positives intra-epoch, every inter-day transition fires), isolation of the proxy-trained $J(G)$ collapse mechanism (OOS ρ 0.818$\rightarrow$0.13, feature ablation and $k \approx$ 100–128 variance crossover), and an end-to-end test showing that at our scale refresh (+33/37 pp, Fisher p<0.02) dominates average placement (bounded null, MDE $\approx$ 32 pp) while placement still sets the reliability floor at $k \geq 9$ under stale pins. An exact reliability composition of the separable software path (App. D) completes the middleware picture. The failures set the command line for future work: extended QPU accumulation beyond D3, refresh-conditional placement ($C(q)$-vs-connectivity trade-off, including the `q11`/`q13` late-chain failures observed even under fresh pins), and active-learning refits of $J(G)$ on real labels across devices. The open repository makes every number traceable to a job or snapshot.

REFERENCES

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.
[2] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.
[3] Y. Kim *et al.*, "Evidence for the utility of quantum computing before fault tolerance," *Nature*, vol. 618, pp. 500–505, 2023.
[4] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," *arXiv:1811.04968*, 2018.
[5] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in *Proc. ASPLOS*, 2019, pp. 1015–1029.
[6] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. ASPLOS*, 2019, pp. 1001–1014.
[7] B. Tan and J. Cong, "Optimal layout synthesis for quantum computing," in *Proc. ICCAD*, 2020, pp. 1–9.
[8] L. Lao and D. E. Browne, "Timing and resource-aware mapping of quantum circuits to heterogeneous MWF-gate superconducting quantum processors," *ACM Trans. Quantum Comput.*, vol. 3, no. 3, 2022.
[9] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," *Phys. Rev. Lett.*, vol. 119, p. 180509, 2017.
[10] E. van den Berg, Z. K. Minev, A. Kandala, and K. Temme, "Probabilistic error cancellation with sparse Pauli–Lindblad models on noisy quantum processors," *Nature Physics*, vol. 19, pp. 1116–1121, 2023.
[11] P. D. Nation *et al.*, "Scalable mitigation of measurement errors on quantum computers," *PRX Quantum*, vol. 5, p. 010326, 2024.
[12] S. Bravyi, S. Shaydulin, S. Hu, and K. Temme, "Mitigating measurement errors in multiqubit experiments," *Phys. Rev. Lett.*, vol. 127, p. 200503, 2021.
[13] N. Kanazawa *et al.*, "Quantum error mitigation," *arXiv:2210.00921*, 2022.
[14] R. LaRose *et al.*, "Mitiq: a software package for error mitigation on noisy quantum computers," *Quantum*, vol. 6, p. 749, 2022.
[15] L. Viola, E. Knill, and S. Lloyd, "Dynamical decoupling of open quantum systems," *Phys. Rev. Lett.*, vol. 82, pp. 2417–2421, 1999.
[16] "Qiskit Runtime primitives," IBM Quantum documentation, [Online]. Available: https://docs.quantum.ibm.com/api/qiskit-ibm-runtime
[17] "IBM Quantum Platform calibration data," [Online]. Available: https://quantum.ibm.com/services/resources
[18] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," *arXiv:2405.08810*, 2024.
[19] NVIDIA, "cuQuantum SDK," [Online]. Available: https://developer.nvidia.com/cuquantum-sdk
[20] Amazon, "Amazon Braket," [Online]. Available: https://aws.amazon.com/braket
[21] Microsoft, "Azure Quantum," [Online]. Available: https://azure.microsoft.com/quantum
[22] M. Schuld and F. Petruccione, *Supervised Learning with Quantum Computers*. Cham: Springer, 2018.
[23] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019.
[24] Y. X. Tan, "UniMind: a middleware framework for bridging classical AI workloads and quantum computing backends," tech. report v2.5, 2026 (companion repo `yxtan5022-maker/unimind-v2`).
[25] UniMind placement/drift datasets (2026-08), `data/` in `yxtan5022-maker/unimind-v2`.

APPENDIX

A. Per-row results of the k=9–18 suite

The 14 circuits (seven sizes × two seeded patterns) with greedy-chain placement, 4096 shots, tolerance 0.05. Worst qubit is the failing or most-off bit; ranks are over all 156 qubits at D1.

TABLE XVII
PER-ROW MAX DEVIATION UNDER FRESH (D1/D2) AND STALE (D0) PINS.

k	pat.	fresh	worst@D1	stale	worst@D1
9	0	.064	q11 (71)	.151	q4 (52)
9	1	.056	q11 (71)	.045	q24 (118)
10	0	.053	q14 (56)	.118	q23 (136)
10	1	.035	q13 (29)	.088	q23 (136)
11	0	.081	q11 (71)	.049	q4 (52)
11	1	.042	q11 (71)	.145	q4 (52)
12	0	.041	q10 (7)	.067	q4 (52)
12	1	.038	q9 (74)	.107	q4 (52)
14	0	.037	q13 (29)	.183	q46 (126)
14	1	.077	q11 (71)	.096	q23 (136)
16	0	.040	q4 (52)	.265	q67 (132)
16	1	.085	q4 (52)	.239	q67 (132)
18	0	.089	q11 (71)	.332	q67 (132)
18	1	.047	q13 (29)	.150	q67 (132)

B. Data and reproducibility

Every table is backed by traced data in the repository: snapshots, E2E batches, real $J(G)$ labels, and the analysis scripts listed below.

```
data/drift/calib_2026-{0829,0830,08-31,0901}.json (D0--D3, 4 daily)
data/calib_snapshots/{marrakesh,kingston,fez}/*.json (337 hourly, 71 epochs)
data/e2e/e2e_angle_{full,ablated}*.json
data/jgpu/j_real*.json
analysis/*.py + analysis/results/*.json
analysis/results/refresh_policy_analysis.json (144 threshold combos)
analysis/results/power_analysis.json (Wilson+Fisher+MDE)
analysis/results/jg_failure_analysis.json + jg_feature_ablation.json + utility_oos_v5.json
analysis/results/ranking_margin_analysis.json + workload_connectivity_drift.json + survival_analysis.json
```

QPU job IDs appear in the file headers: `daadeocjbipc73ffq83g` (refresh-full E2E), `daadeosjbipc73ffq840` (refresh-ablated E2E), and `daadgmmrbfbs73cijmbg` (refresh-pinned $J(G)$ labels). Hourly telemetry is calibration-only (no QPU jobs). Transpiler seed 42 everywhere; shots per experiment as stated. All statistical tests are two-sided Fisher exact unless noted; Wilson 95% CIs; MDE at 80% power reported for bounded nulls.

C. Calibration-weighted routing benchmark

Interactive (local-simulator, zero-quota) comparison on snapshot 2026-08-29 (156 qubits, 176 couplers; RY–CX–RY, opt=1, seed 42). UniMind computes a $C(q) + s_x$-weighted SABRE candidate set and keeps the better of weighted and global SABRE, guaranteeing no regression for $k \geq 8$ (violation check: none).

TABLE XVIII
PLACEMENT COST OF THE FIVE STRATEGIES (SUMMARY ROWS; FULL PER-k TABLES IN THE REPO).

k	2	4	6	8	10	16
Random SWAPs	8	42	68	105	118	207
UniMind SWAPs	0	3	8	20	20	50
Δ SWAP vs. random	-100%	-93%	-88%	-81%	-83%	-76%
Random 2q err	.118	.484	.666	.821	.847	.979
UniMind 2q err	.002	.049	.065	.133	.181	.353
Δ vs. Default	$=0$	$=0$	$=0$	$=0$	$=0$	$=0$

D. Orchestration reliability calculus and Unibit instrument (separable)

The LLM-orchestration layer is *not* evaluated on QPU fidelity; it is a controlled-parameter mock (`analysis/mock_llm.py`, `reliability_model.py`) whose two parameters (q, f) are swept ($q \in \{0.5, 0.7, 0.9, 1.0\}$, availability stress $f \in \{0.1, 0.3, 0.5\}$) and whose exact decomposition (Eqs. 6–8, H=4, K=3, abort-on-`None`) is validated cell-by-cell (23/23 inside Wilson 95% CI, Table XII). The naive independence baseline $1 - (1 - q)^4$ overestimates by 7.8–12.4 pp; the structural gap is a retry-budget truncation effect, not correlation. Full derivation, ablation grids, and the Unibit preprocessing invariants (13 property tests, sinc smoothing, ϕ=0.42) are in `analysis/reliability_instrumented.json`, `ablation_*.json`, and `unibit_math.json`—included for completeness so the scheduling core (§IV-B–§VII-E) can be evaluated independently.